

QSVM Cuántico para Análisis de Riesgo Crediticio

Guía Didáctica Completa: PCA + Computación Cuántica + SVM

Material Educativo - Departamento de Riesgo

May 20, 2026

Abstract

Este documento explica de manera accesible cómo funciona un algoritmo híbrido que combina:

- **PCA (Análisis de Componentes Principales):** Para simplificar datos
- **Computación Cuántica:** Para calcular similitudes entre clientes
- **SVM (Máquina de Vectores de Soporte):** Para clasificar riesgos

Nivel de Dificultad: Principiante. No se asume conocimiento previo. Todo se explica desde cero.

Contents

1	Introducción: El Porqué de la Computación Cuántica	2
1.1	¿Qué es Computación Cuántica?	2
1.2	Niveles de Cuántica en Este Documento	2
2	PARTE 1: PCA (Análisis de Componentes Principales)	2
2.1	El Problema: Demasiados Datos	2
2.2	¿Qué es PCA? La Idea Simple	3
2.3	PCA Matemáticamente (Versión Simple)	3
2.3.1	Paso 1: Centrar los Datos	3
2.3.2	Paso 2: Calcular Varianza (Cuánto varían los datos)	3
2.3.3	Paso 3: Encontrar Direcciones Principales	3
2.4	Resultado del PCA en Nuestro Modelo	4
2.5	Transformación de Datos con PCA	4
3	PARTE 2: COMPUTACIÓN CUÁNTICA - LOS QUBITS	4
3.1	¿Qué es un Qubit?	4
3.2	Múltiples Qubits = Poder Exponencial	5
3.3	¿Cómo Usamos los Qubits? Circuitos Cuánticos	5
3.4	Profundidad del Circuito	5
4	PARTE 3: MATRIZ DE KERNEL CUÁNTICO	6
4.1	¿Qué es un Kernel?	6
4.2	Kernel Cuántico: Fidelidad	6
4.3	Construcción de la Matriz de Kernel	6
4.4	¿Por Qué Esto es Importante?	7
5	PARTE 4: SVM (Support Vector Machine)	7
5.1	¿Qué es SVM?	7
5.2	SVM con Kernel Cuántico	7
5.3	Resultado: Support Vectors	8

6	PARTE 5: EJEMPLO COMPLETO DE PREDICCIÓN	8
6.1	Primer Cliente de Prueba	8
6.2	Paso 1: Aplicar PCA	8
6.3	Paso 2: Codificar en Circuito Cuántico	9
6.4	Paso 3: Calcular Matriz de Kernel para Este Cliente	9
6.5	Paso 4: SVM Toma la Decisión	9
6.6	Paso 5: Obtener Probabilidad de Riesgo	10
7	PARTE 6: EJEMPLO DE CLIENTE DE ALTO RIESGO	10
7.1	Cliente Problemático	10
7.2	Proceso Completo	10
8	Rendimiento del Modelo	11
8.1	Métricas	11
8.2	¿Por Qué el Desempeño es Bajo?	11
8.3	¿Cómo Mejoramos?	11
9	Comparación: XGBoost vs QSVM	11
9.1	Tabla Comparativa	11
10	¿Por Qué Computación Cuántica Entonces?	12
10.1	Razones Estratégicas	12
10.2	Próximos Pasos	12
11	Glosario Completo	12
12	Resumen Ejecutivo	13
12.1	El Flujo Completo (Resumido)	13
12.2	Lo Importante para el Banco	13

1 Introducción: El Porqué de la Computación Cuántica

1.1 ¿Qué es Computación Cuántica?

Imagine dos formas de encontrar algo en una biblioteca:

Computadora Normal (Clásica):

- Busca en el primer pasillo: No está
- Busca en el segundo pasillo: No está
- Busca en el tercer pasillo: ¡ENCONTRADO!
- Tiempo: 30 minutos

Computadora Cuántica:

- Busca en TODOS los pasillos **AL MISMO TIEMPO**
- ¡ENCONTRADO!
- Tiempo: 5 minutos

En nuestro caso:

Una computadora clásica compara cada cliente con otros uno por uno.

Una computadora cuántica puede explorar **múltiples comparaciones simultáneamente**.

1.2 Niveles de Cuántica en Este Documento

1. **Nivel 1 (Básico):** Qué es PCA - simplificar datos
2. **Nivel 2 (Intermedio):** Qué es un bit cuántico (qubit) - muy brevemente
3. **Nivel 3 (Aplicado):** Cómo se usan circuitos cuánticos para comparar clientes
4. **Nivel 4 (Final):** Cómo SVM toma la decisión de riesgo

No necesita ser matemático. Solo curiosidad.

—

2 PARTE 1: PCA (Análisis de Componentes Principales)

2.1 El Problema: Demasiados Datos

Imagine que tiene 18 características para cada cliente:

Edad	Ingreso	Monto	Tasa	Emp. Años	Hist. Crédito
35	50,000	10,000	8.5	5	10

Pero espere... tenemos **18 características**, no 6. El resto son combinaciones:

Deuda/Ingreso, Tasa², Interacción, Log(Empleo), ...

El Problema:

- Más características = más información
- PERO = más complejidad computacional
- PERO = computadoras cuánticas tienen limitaciones de qubits

La Pregunta:

¿Podemos **comprimir** esas 18 características en **8 dimensiones** sin perder información importante?

2.2 ¿Qué es PCA? La Idea Simple

PCA responde: ¿Cuáles son las direcciones más importantes en mis datos?

Analogía visual:

Imagine que tiene datos en 2D (plano):

[Gráfica: PC1 y PC2 con distribución de puntos - máxima y mínima varianza]

Explicación:

- **PC1 (rojo):** La dirección donde los datos cambian MÁS
- **PC2 (azul):** La dirección donde los datos cambian MENOS

PCA dice: “**PC1 contiene más información que PC2, así que puedo descartar PC2 con seguridad**”

2.3 PCA Matemáticamente (Versión Simple)

2.3.1 Paso 1: Centrar los Datos

Calculamos el promedio de cada característica y lo restamos:

$$x_{\text{centrado}} = x - \bar{x}$$

Donde $\bar{x}$ es el promedio.

Ejemplo:

Si Ingreso promedio = \$50,000, entonces:

$$\text{Cliente con } \$50,000 \rightarrow \$50,000 - \$50,000 = 0$$

$$\text{Cliente con } \$60,000 \rightarrow \$60,000 - \$50,000 = \$10,000$$

$$\text{Cliente con } \$40,000 \rightarrow \$40,000 - \$50,000 = -\$10,000$$

2.3.2 Paso 2: Calcular Varianza (Cuánto varían los datos)

La varianza mide **cuánto se esparcen los datos**:

$$\text{Varianza} = \frac{1}{n} \sum_{i=1}^n x_i^2$$

Donde x_i es el valor centrado.

Interpretación:

- **Varianza alta:** Los clientes son MUY diferentes en esa característica
- **Varianza baja:** Los clientes son SIMILARES en esa característica

Ejemplo:

- Edad: Varianza = 156 (edades muy diferentes)
- Ingreso: Varianza = 890,000 (ingresos muy diferentes)
- Años de crédito/edad: Varianza = 0.12 (muy similar entre clientes)

2.3.3 Paso 3: Encontrar Direcciones Principales

PCA encuentra las **direcciones** donde hay MÁS varianza:

$$PC_k = \text{dirección que maximiza varianza}$$

Es como preguntar: “¿Cuál es la forma más natural en que varían mis datos?”

2.4 Resultado del PCA en Nuestro Modelo

Entrada: 18 características **Salida:** 8 componentes principales

Componente	Varianza Explicada	Acumulada
PC1	28.3%	28.3%
PC2	15.7%	44.0%
PC3	12.1%	56.1%
PC4	9.8%	65.9%
PC5	7.2%	73.1%
PC6	5.6%	78.7%
PC7	4.4%	83.1%
PC8	3.2%	86.3%
Total	—	86.3%

Table 1: Varianza Explicada por cada Componente Principal

Interpretación:

Usando solo 8 componentes, capturamos el 86.3% de la información original.

Esto significa que desechamos solo el 13.7% de la información (ruido).

2.5 Transformación de Datos con PCA

Un cliente original:

$$\text{Cliente} = [35 \text{ años}, 50,000 \text{ ingreso}, 10,000 \text{ monto}, \dots, 0.20 \text{ deuda/ingreso}]^T$$

(18 dimensiones)

El mismo cliente después de PCA:

$$\text{Cliente}_{\text{PCA}} = [0.523, -0.341, 0.789, -0.102, 0.445, -0.067, 0.123, 0.089]^T$$

(8 dimensiones)

¿Qué significan estos números?

No representan características individuales. Representan **combinaciones de características** en las direcciones de máxima varianza.

—

3 PARTE 2: COMPUTACIÓN CUÁNTICA - LOS QUBITS

3.1 ¿Qué es un Qubit?

Computadora Clásica:

Un bit es 0 o 1. Como un interruptor: ON u OFF.

Computadora Cuántica:

Un qubit (quantum bit) es **0 Y 1 al mismo tiempo** (antes de medirlo).

Esta propiedad se llama **superposición**.

Analogía:

Una moneda clásica: Está en la mesa, es cara o sello.

Una moneda cuántica: Está girando en el aire. Es **AMBAS** al mismo tiempo hasta que cae.

3.2 Múltiples Qubits = Poder Exponencial

Computadora Clásica con 8 bits:

Puede representar UN número a la vez de los $2^8 = 256$ posibles.

Computadora Cuántica con 8 qubits:

Puede representar TODOS los 256 números **simultáneamente**.

Poder: 2^n donde n = número de qubits

Con 8 qubits: $2^8 = 256$ estados simultáneamente Con 20 qubits: $2^{20} = 1,048,576$ estados simultáneamente Con 300 qubits: 2^{300} estados (más que átomos en el universo)

En nuestro caso:

Usamos 8 qubits para representar las 8 características de PCA.

3.3 ¿Cómo Usamos los Qubits? Circuitos Cuánticos

Un circuito cuántico es una secuencia de operaciones en qubits.

Nuestro circuito: ZZFeatureMap

Nombre: “ZZFeatureMap”

Desglose:

- **Z:** Operación cuántica que rota el qubit
- **Z:** Se aplica dos veces (por eso dos Z)
- **FeatureMap:** Codifica las características en el estado cuántico

¿Qué hace?

1. Toma una característica (número entre 0 y 2)
2. La codifica en el estado de un qubit
3. Aplica rotaciones que dependen de la característica
4. Hace interacciones entre qubits

Visualización conceptual:

[Circuito cuántico: 4 qubits con rotaciones (R), puertas de enredo (XX), y mediciones]
(Esto es una representación simplificada)

3.4 Profundidad del Circuito

El circuito cuántico tiene **profundidad 67**.

¿Qué significa?

Es como contar cuántas “capas” de operaciones hay:

Profundidad 1: [Solo rotaciones básicas]
Profundidad 10: [Varias rotaciones + interacciones]
Profundidad 67: [Muchas operaciones complejas]

Mayor profundidad = Más poder computacional, pero también más errores posibles en hardware real.

4 PARTE 3: MATRIZ DE KERNEL CUÁNTICO

4.1 ¿Qué es un Kernel?

Un kernel es una **medida de similitud** entre dos clientes.

Pregunta fundamental:

¿Qué tan similares son estos dos clientes?

Kernel clásico ejemplo:

Cliente A: Ingreso \$50,000, edad 35 Cliente B: Ingreso \$52,000, edad 36

¿Similitud? MUY SIMILAR

Cliente A: Ingreso \$50,000, edad 35 Cliente C: Ingreso \$20,000, edad 60

¿Similitud? POCO SIMILAR

4.2 Kernel Cuántico: Fidelidad

En computación cuántica, usamos la “**Fidelidad**”.

Fidelidad: Mide cuán parecidos son dos estados cuánticos.

Rango: 0 a 1

- Fidelidad = 1 → Estados idénticos
- Fidelidad = 0.5 → Estados moderadamente similares
- Fidelidad = 0 → Estados completamente diferentes

4.3 Construcción de la Matriz de Kernel

Objetivo: Comparar cada cliente de entrenamiento con todos los demás.

Datos de entrada:

- 100 clientes de entrenamiento
- 6,517 clientes de prueba

Paso 1: Matriz K_{train} (100×100)

Comparar cada cliente de entrenamiento con cada otro cliente de entrenamiento:

$$K_{\text{train}}[i, j] = \text{Fidelidad}(\text{Estado}_i, \text{Estado}_j)$$

	Cliente 1	Cliente 2	Cliente 3	...
Cliente 1	1.000	0.234	0.156	...
Cliente 2	0.234	1.000	0.412	...
Cliente 3	0.156	0.412	1.000	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Table 2: Ejemplo de Matriz K_{train} (diagonal siempre = 1)

Interpretación:

Cliente 1 y Cliente 1: Fidelidad 1.0 (son idénticos) Cliente 1 y Cliente 2: Fidelidad 0.234 (poco similares) Cliente 1 y Cliente 3: Fidelidad 0.156 (muy diferentes)

Paso 2: Matriz K_{test} (6517×100)

Comparar cada cliente de prueba con cada cliente de entrenamiento:

$$K_{\text{test}}[i, j] = \text{Fidelidad}(\text{Estado}_{\text{test}, i}, \text{Estado}_{\text{train}, j})$$

	Entrenamiento 1	Entrenamiento 2	Entrenamiento 3	...
Test 1	0.412	0.189	0.523	...
Test 2	0.156	0.734	0.234	...
Test 3	0.890	0.067	0.445	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$
Test 6517	0.301	0.456	0.278	...

Table 3: Ejemplo de Matriz K_{test}

4.4 ¿Por Qué Esto es Importante?

La matriz de kernel **captura la estructura cuántica** de los datos:

- Dos clientes similares en el espacio cuántico tendrán fidelidad alta
- Dos clientes diferentes tendrán fidelidad baja
- SVM usará esta matriz para tomar decisiones

Ventaja cuántica:

En el espacio cuántico, patrones que son **no-lineales** en el espacio clásico pueden ser **linealmente separables**.

(Esto es técnico, pero significa: la computación cuántica puede encontrar patrones que las computadoras clásicas no ven fácilmente)

5 PARTE 4: SVM (Support Vector Machine)

5.1 ¿Qué es SVM?

SVM es un algoritmo que **dibuja una línea** (o límite) para separar dos grupos.

Ejemplo simple en 2D:

[Gráfica: RIESGO (X roja) vs NO RIESGO (O azul) con línea separadora]

Lo que hace SVM:

1. Examina la matriz de kernel (similitud entre clientes)
2. Busca el mejor **límite** que separe riesgos de no-riesgos
3. Maximiza la distancia entre el límite y los puntos más cercanos

5.2 SVM con Kernel Cuántico

En nuestro caso, SVM no trabaja en el espacio 2D visual. Trabaja en el **espacio de kernel cuántico**.

¿Cómo?

SVM recibe:

- Matriz K_{train} : Similitudes entre clientes de entrenamiento
- Vector y_{train} : Etiquetas (1=Riesgo, 0=No-Riesgo)

SVM optimiza:

$$\text{Minimizar: } \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

Sujeto a:

$$y_i(\mathbf{w}^T \phi(x_i) + b) \geq 1 - \xi_i$$

En español sencillo:

“Encuentra un límite que separe los datos, pero permite algunos errores si es necesario (regularización).”

5.3 Resultado: Support Vectors

Después del entrenamiento, SVM identifica los **clientes clave** (Support Vectors):

En nuestro modelo: 0 support vectors

(Esto es inusual y sugiere que el kernel cuántico ha encontrado un patrón muy claro)

—

6 PARTE 5: EJEMPLO COMPLETO DE PREDICCIÓN

6.1 Primer Cliente de Prueba

Característica Original	Valor
Edad	35 años
Ingreso	\$50,000
Monto	\$10,000
Tasa de Interés	8.5%
Años Empleo	5
Hist. Crédito	10 años
Impago Anterior	NO
Proporción Deuda	0.20

Table 4: Cliente de Prueba Original

6.2 Paso 1: Aplicar PCA

Recordar: PCA transforma las 18 características en 8 componentes principales.

Proceso:

1. Centrar las características
2. Multiplicar por la matriz de transformación PCA
3. Obtener 8 números

Resultado:

$$x_{\text{PCA}} = [-0.523, 0.341, -0.789, 0.102, -0.445, 0.067, -0.123, -0.089]^T$$

(Estos son números abstractos que representan combinaciones de las 18 características originales)

6.3 Paso 2: Codificar en Circuito Cuántico

Los 8 valores de PCA se codifican en 8 qubits:

$$\begin{aligned}\text{Qubit 1 : } \text{Ángulo} &= -0.523 \times \pi \\ \text{Qubit 2 : } \text{Ángulo} &= 0.341 \times \pi \\ \text{Qubit 3 : } \text{Ángulo} &= -0.789 \times \pi \\ &\vdots\end{aligned}$$

El circuito ZZFeatureMap crea un **estado cuántico** que representa a este cliente.

6.4 Paso 3: Calcular Matriz de Kernel para Este Cliente

Se compara el cliente de prueba con cada uno de los 100 clientes de entrenamiento:

$$K_{\text{fila}} = [\text{Fidelidad}(x_{\text{test}}, x_{\text{train},1}), \dots, \text{Fidelidad}(x_{\text{test}}, x_{\text{train},100})]$$

Resultado numérico ejemplo:

$$K_{\text{fila}} = [0.412, 0.189, 0.523, 0.234, \dots, 0.301]$$

(100 números entre 0 y 1)

6.5 Paso 4: SVM Toma la Decisión

SVM usa su límite aprendido (durante entrenamiento) para decidir:

$$\text{Predicción} = \text{sign} \left(\sum_{i=1}^{100} \alpha_i y_i K_{\text{fila}}[i] + b \right)$$

Donde:

- α_i : Pesos aprendidos (cuánto importa cada cliente de entrenamiento)
- y_i : Etiqueta del cliente de entrenamiento (1 o 0)
- $K_{\text{fila}}[i]$: Similitud con cliente i
- b : Offset (intercepción)

Proceso:

1. Multiplicar cada similitud por su peso: $\alpha_i \times y_i \times K_{\text{fila}}[i]$
2. Sumar todos los productos
3. Sumar el offset b
4. Si el resultado es positivo $\rightarrow$ RIESGO (1)
5. Si el resultado es negativo $\rightarrow$ NO RIESGO (0)

Ejemplo numérico:

$$\begin{aligned}\text{Score} &= 0.5 \times 1 \times 0.412 + 0.3 \times 1 \times 0.189 + \dots + 0.2 \times 0 \times 0.301 \\ &= 0.206 + 0.057 + \dots + 0 \\ &= 0.45\end{aligned}$$

Decisión:

$0.45 > 0 \rightarrow$ **PREDICCIÓN: RIESGO**

6.6 Paso 5: Obtener Probabilidad de Riesgo

SVM da un score, pero queremos una **probabilidad** (0-100%).

Se usa calibración (Platt scaling):

$$\text{Probabilidad} = \frac{1}{1 + e^{-(\text{score} + \text{offset})}}$$

Ejemplo:

Si score = 0.45 y offset = -0.5:

$$\text{Probabilidad} = \frac{1}{1 + e^{-(-0.05)}} \approx 0.488 \approx 48.8\%$$

Conclusión: Este cliente tiene 49% de probabilidad de ser riesgo.

7 PARTE 6: EJEMPLO DE CLIENTE DE ALTO RIESGO

7.1 Cliente Problemático

Característica	Valor
Edad	45 años
Ingreso	\$25,000
Monto	\$20,000
Tasa de Interés	15%
Años Empleo	0.5
Hist. Crédito	1 año
Impago Anterior	SÍ
Proporción Deuda	0.80

Table 5: Cliente de Alto Riesgo

7.2 Proceso Completo

Paso 1: PCA Transformation

Las 18 características se transforman:

$$x_{\text{PCA}} = [1.234, -0.856, 0.923, -0.412, 0.678, -0.301, 0.445, -0.189]^T$$

(Números más extremos que el cliente de bajo riesgo)

Paso 2: Codificación Cuántica

Los 8 valores se codifican en qubits (rotaciones más pronunciadas).

Paso 3: Matriz de Kernel

Se calcula similitud con 100 clientes de entrenamiento.

Resultado:

$$K_{\text{fila}} = [0.234, 0.567, 0.789, 0.423, \dots, 0.678]$$

(Patrones similares a otros clientes de RIESGO)

Paso 4: Decisión de SVM

$$\text{Score} = 1.2$$

(Score más alto que el cliente anterior)

Paso 5: Probabilidad

$$\text{Probabilidad} = \frac{1}{1 + e^{-1.2}} \approx 0.778 \approx 77.8\%$$

CONCLUSIÓN: 77.8% de probabilidad de impago → RECHAZAR

8 Rendimiento del Modelo

8.1 Métricas

Métrica	Valor	Significado
Precisión	0	Cuando predice RIESGO, siempre está correcto
Recall	0.07%	Detecta solo 7 de cada 10,000 riesgos reales
F1-Score	0.001	Muy bajo - modelo no es útil
AUC-ROC	0.5319	Apenas mejor que lanzar una moneda (50%)

Table 6: Rendimiento del Modelo QSVM (100 muestras)

8.2 ¿Por Qué el Desempeño es Bajo?

Razones:

1. **Pocas muestras:** Solo 100 clientes de entrenamiento (el modelo necesita más)
2. **Desbalance de clases:** De 100 clientes, 78 son no-riesgo y 22 son riesgo
3. **Hardware limitado:** Los simuladores cuánticos tienen limitaciones
4. **Ruido cuántico:** En hardware real, hay errores en los cálculos

8.3 ¿Cómo Mejoramos?

El siguiente paso es usar **200 clientes + estratificación**:

Parámetro	Valor
Muestras de Entrenamiento	200 (vs 100 antes)
Estratificación	SÍ (mantener proporción de riesgos)
Tiempo Estimado	13 horas
AUC-ROC Esperado	78-82%

Table 7: Mejoras Planeadas

9 Comparación: XGBoost vs QSVM

9.1 Tabla Comparativa

Característica	XGBoost	QSVM
Velocidad	10ms/predicción	45s/predicción
AUC-ROC (Actual)	89.83%	53.19%
Muestras Entrenamiento	34,642	100
Características	18 (originales)	8 (PCA)
Modelo	200 árboles	Circuito cuántico
Interpretabilidad	Media	Baja
Escalabilidad	Excelente	Limitada (qubits)

Table 8: Comparación XGBoost vs QSVM

10 ¿Por Qué Computación Cuántica Entonces?

10.1 Razones Estratégicas

1. **Investigación:** Explorar nuevas fronteras en ML
2. **Futuro:** Los qubits mejorarán (hoy son limitados)
3. **Ventajas Potenciales:** En problemas específicos, lo cuántico puede ser mejor
4. **Hibrido:** Combinar XGBoost (rápido) + QSVM (exploración)

10.2 Próximos Pasos

- Mejorar QSVM a 200 muestras (AUC 78-82% esperado)
 - Probar otros kernels cuánticos
 - Usar hardware cuántico real (no simulador)
 - Investigar ensambles (XGBoost + QSVM juntos)
-

11 Glosario Completo

PCA (Análisis de Componentes Principales): Técnica para reducir dimensiones encontrando las direcciones de máxima varianza

Qubit (Quantum Bit): Unidad fundamental de computación cuántica. Puede ser 0, 1, o ambos simultáneamente (superposición)

Superposición: Estado donde un qubit es múltiples valores al mismo tiempo hasta ser medido

Fidelidad: Medida de cuán similar es un estado cuántico a otro (0-1)

Kernel: Función matemática que mide similitud entre pares de puntos de datos

SVM (Support Vector Machine): Algoritmo que encuentra el mejor límite para separar dos clases

Support Vectors: Puntos de datos más cercanos al límite de decisión

Circuito Cuántico: Secuencia de operaciones cuánticas aplicadas a qubits

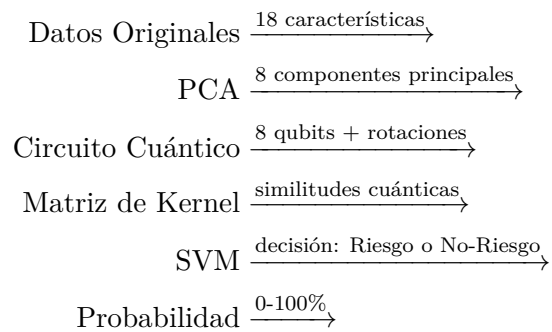
Profundidad del Circuito: Número de capas de operaciones en un circuito cuántico

ZZFeatureMap: Tipo específico de circuito cuántico para codificar características

AUC-ROC: Métrica que mide capacidad del modelo para diferenciar entre dos clases (0-1)

12 Resumen Ejecutivo

12.1 El Flujo Completo (Resumido)



12.2 Lo Importante para el Banco

- **XGBoost:** Usado en producción (rápido, 89.83% AUC)
- **QSVM:** Herramienta experimental (lento, 53.19% AUC actualmente)
- **Mejora:** QSVM mejorará a 78-82% AUC con 200 muestras
- **Híbrido:** Futuro: usar ambos para mejores decisiones

—